



UNIVERSIDADE
FEDERAL DO CEARÁ

Discentes

Ana Beatriz Leite Damascena
Cícero Rodrigues da Silva Neto

Docente: Rafael Braga
Disciplina: Sistemas Distribuídos

Relatório de Implementação: Trabalho 3 - Web Services e API REST
08/06/2026

Quixadá - CE

1. Introdução

O presente relatório descreve a arquitetura e o desenvolvimento da terceira entrega da disciplina de Sistemas Distribuídos. O objetivo central deste projeto consistiu na evolução do sistema de atendimento de telecomunicações desenvolvido nas etapas anteriores, substituindo o *middleware* de Invocação Remota de Método (RMI) por uma arquitetura baseada em Web Services e API REST.

Atendendo rigorosamente às especificações do projeto, a comunicação via rede abandonou o uso de Sockets nativos e do protocolo RMI, adotando o protocolo HTTP estruturado no modelo Cliente-Servidor. Além disso, a fim de comprovar a interoperabilidade da arquitetura REST, os clientes foram desenvolvidos utilizando linguagens de programação distintas daquela empregada no servidor, e os testes foram conduzidos em um ambiente de rede real (LAN/VPN) com computadores distintos.

2. Arquitetura do Sistema e Tecnologias

A transição para o padrão arquitetural REST proporcionou um alto grau de desacoplamento e interoperabilidade ao sistema. A aplicação foi dividida em dois componentes principais: o Servidor Central e os Clientes Heterogêneos.

2.1. O Servidor (Java + Spring Boot)

O núcleo do sistema foi implementado em Java utilizando o *framework* Spring Boot. A lógica de negócios (*ReclamacaoServiceImpl*) e as entidades de domínio previamente modeladas (*Linha*, *Servico*, *ClienteEmpresa*) foram integralmente preservadas, demonstrando a manutenibilidade do código.

A camada de rede manual (o antigo *GatewayRMI*) foi substituída por um controlador REST (*TelecomController*). Este componente é responsável por expor os *endpoints* HTTP e gerenciar a conversão de e para o formato JSON de forma nativa e transparente.

2.2. Os Clientes (Poliglotismo)

Para satisfazer a restrição de utilização de pelo menos duas linguagens distintas da linguagem do servidor, foram desenvolvidos dois clientes independentes:

- **Cliente 1 (Python):** Utiliza a biblioteca *requests* para construir e disparar as requisições HTTP de forma síncrona. A interação com o usuário ocorre via interface de terminal (CLI).
- **Cliente 2 (JavaScript / Node.js):** Emprega a API nativa *fetch* para o envio das requisições HTTP e o módulo *readline* para capturar as entradas do usuário, provendo uma interface de terminal assíncrona e orientada a eventos.

3. Comunicação e Fluxo de Execução

O modelo de comunicação adotou o formato de Representação Externa de Dados em JSON sobre o protocolo HTTP. O ciclo de vida de uma operação (como o registro de uma reclamação) segue o seguinte fluxo:

1. **Interação e Captura:** O usuário interage com o menu CLI no cliente (seja em Python ou Node.js) e insere os dados pertinentes.
2. **Requisição HTTP:** O cliente serializa os dados e constrói uma requisição HTTP (utilizando os verbos semânticos adequados, como POST para criação e GET para consulta) direcionada ao *endpoint* do servidor (ex: */api/telecom/reclamacoes*).
3. **Desserialização e Roteamento:** O servidor *web* embutido (Tomcat) intercepta a requisição. O Spring Boot realiza a desserialização automática do *body* da requisição de JSON para instâncias de objetos Java e invoca o método roteado no *TelecomController*.
4. **Processamento da Regra de Negócio:** O controlador repassa a chamada para a camada de serviço em memória, executando operações como a geração de protocolos UUID e consultas em mapas de dispersão (HashMap).
5. **Resposta HTTP:** Após a conclusão, o servidor envelopa o resultado e o retorna acompanhado de um código de *status* HTTP adequado (como 200 OK ou 201 *Created*). O cliente processa este JSON e exibe o resultado formatado no console do usuário.

4. Implantação e Validação Distribuída

O sistema foi validado em um cenário de rede física distribuída, simulando um ambiente de produção real. A execução ocorreu da seguinte forma:

- **Hospedagem do Servidor:** O servidor Spring Boot foi executado em uma máquina anfitriã, tornando-se acessível na porta 8080 de sua interface de rede local (LAN).
- **Acesso dos Clientes:** Nos computadores clientes, a variável de ambiente ou constante `BASE_URL` (presente em `cliente.py` e `cliente.js`) foi configurada para apontar explicitamente para o endereço IP do servidor hospedeiro.

A comunicação transcorreu de forma bem-sucedida, comprovando que requisições originadas por interpretadores diferentes (Python e Node V8) puderam interagir com a mesma base de dados volátil mantida pela Máquina Virtual Java (JVM).

5. Conclusão

A implementação deste projeto demonstrou na prática as vantagens fundamentais do padrão REST em Sistemas Distribuídos. Ao abdicar do RMI — que acopla fortemente a aplicação ao ecossistema Java — e adotar *endpoints* HTTP padronizados consumindo JSON, o sistema alcançou verdadeira interoperabilidade. Foi possível construir clientes em linguagens completamente distintas consumindo os mesmos recursos do servidor sem a necessidade de reescrever a lógica de domínio, cumprindo de forma plena os requisitos acadêmicos da disciplina.